

100 Days of Azure DevOps

Day 12 Handout — Git Flow is a seating chart. Trunk-based is a food truck. Pick for ship cadence.

2 - Azure Repos & Git Mastery · Personal lab only · Educational content · Not a sales pitch

Architecture A — three models (same Git, different branch lifetime)

Model	Refs you keep	Merge / release
Git Flow	main + develop + feature/* + release/* + hotfix/*	release/* cut; merge to main AND develop
GitHub Flow	main + short feature/*	PR to main; deploy main
Trunk-based	main; branches hours not days; maybe release/* tags	Small PRs; feature flags if needed
This lab	main protected (Day 19) + feature/* < 2 days	Squash PR; no develop branch
TFVC / release branches forever	long-lived release/2024.09	Out of scope — you stay on Git
Environment branches (dev/qa/prod)	Three eternally diverging mains	Avoid; promote artifacts not branches

Architecture B — choose by constraint (not by blog title)

Constraint	Default for this series	Choose Git Flow when
Ship cadence	GitHub Flow / trunk-lite: many small PRs	Hard monthly cut with 2+ supported versions
Team size	1 person + future PR policy	Large role-split (dev vs release managers)
CI maturity	Need YAML on main (Day 21+)	You cannot run CI on every feature branch
Hotfix	PR straight to main, then patch	You already maintain release/* lines
DORA lead time	Hours: feature/* lived < 2 days	Weeks: develop is a parking lot
Azure Repos policies	Require PR to main (Day 19)	Policies on develop AND main (double gates)

Architecture C — pitfalls that recreate Git Flow by accident

Pitfall	What you see	Fix
develop 'just in case'	PRs to develop; main is months behind	Delete develop; protect main only
feature/* weeks long	Merge conflicts vs 40 files	Integrate daily; smaller stories on Boards
Environment branches	dev/qa/prod all have unique commits	One main; pipelines deploy by stage
Hotfix off an old SHA you cannot find	git log archaeology at 2am	Tag releases; hotfix from that tag
No ADR	Slack argument next month	docs/branching-strategy.md in the repo
Force-push main	Rewritten SHAs, broken pipelines	Deny force push (Day 17/19)

One-liner to remember

Git Flow is a wedding seating chart; trunk-based is a food truck — pick complexity that matches how often you ship.

Step-by-step lab (personal Azure / Azure DevOps only)

Personal azure-100-labs. Write an ADR. Do not create a develop branch 'for realism'.

- 1 `git switch main` && `git pull` && `git switch -c feature/day12-branching-adr`
- 2 Create docs/branching-strategy.md with Decision: GitHub Flow (trunk-based lite).
- 3 Document: main always deployable; feature/* < 2 days; PR required; squash merge; no develop/release/hotfix long lives.
- 4 List 3 reasons Git Flow is rejected for this lab (solo, no multi-version prod, want short lead time).
- 5 `git add docs/branching-strategy.md` && `git commit -m "docs: add branching strategy ADR"` && `git push -u origin feature/day12-branching-adr`
- 6 Azure Repos → Pull requests → New PR into main. Title: docs: branching ADR. You may complete it or leave for Day 14 template.
- 7 Repos → Branches. Confirm there is no develop. If you created one by habit, delete it after merging any needed commits.

ADR skeleton committed on a short-lived feature branch

```
# docs/branching-strategy.md
# Decision: GitHub Flow (trunk-based lite)
# - main is always deployable
# - feature/* lives < 2 days
# - PR required; squash merge
# - No long-lived develop, release/*, or hotfix/* in this lab
# - Environment promotion happens in pipelines, not extra branches
```

Done checklist

- Can contrast Git Flow vs GitHub Flow vs trunk-based with branch names
- docs/branching-strategy.md is on a feature branch (or merged)
- No develop branch in azure-100-labs
- ADR says squash + short feature/*

Tomorrow — Day 13

Azure Repos setup